

Unsupervised Learning – Take-Home Assignment

Student Name: P.Annapurna

Course: Unsupervised Learning

Assignment: Clustering Foundations & Business Segmentation | Dimensionality Reduction & Embedding Foundations

Tools Used

- Python
- NumPy
- Pandas
- Scikit-learn
- Matplotlib

SECTION A — Output Prediction

A1. K-Means by Hand

Prediction Before Running the Code

We are given six one-dimensional data points:

$X = [1, 2, 3, 10, 11, 12]$

The initial cluster centres are:

$C1 = 1$ $C2 = 2$

There are two clusters, and the initial centres are deliberately close together. Therefore, during the first iteration, most of the points will initially be assigned to the second cluster. After updating the centres, K-Means should separate the small values (1, 2, 3) from the large values (10, 11, 12).

A1(a) Prediction of Line 1: labels_

Initial Centres

$C1 = 1$

$C2 = 2$

Iteration 1 — Assignment

Using centres 1 and 2, the nearest-centre assignments are:

Cluster 1 = [1]

Cluster 2 = [2, 3, 10, 11, 12]

Iteration 1 — Update

$C1 = 1$

$C2 = (2 + 3 + 10 + 11 + 12) / 5 = 7.6$

Therefore:

$C1 = 1$

$C2 = 7.6$

Iteration 2 — Assignment

Using the new centres 1 and 7.6:

Cluster 1 = [1, 2, 3]

Cluster 2 = [10, 11, 12]

Iteration 2 — Update

$C1 = (1 + 2 + 3) / 3 = 2$

$C2 = (10 + 11 + 12) / 3 = 11$

Therefore, the final clusters are:

Cluster 1 = [1, 2, 3]

Cluster 2 = [10, 11, 12]

Since scikit-learn represents the two clusters using labels 0 and 1:

Prediction

[0 0 0 1 1 1]

A1(b) Prediction of Line 2: cluster_centers_

From the previous iterations, the final clusters are:

Cluster 1 = [1, 2, 3]

Cluster 2 = [10, 11, 12]

The centre of each cluster is calculated as the mean of the points belonging to that cluster.

Cluster 1 Centre

$$C1 = (1 + 2 + 3) / 3$$

$$C1 = 6 / 3$$

$$C1 = 2$$

Cluster 2 Centre

$$C2 = (10 + 11 + 12) / 3$$

$$C2 = 33 / 3$$

$$C2 = 11$$

Therefore, the final cluster centres are:

$$C1 = 2$$

$$C2 = 11$$

Prediction

[2. 11.]

A1(c) Prediction of Line 3: inertia_

From the previous calculations, the final clusters are:

Cluster 1 = [1, 2, 3] Cluster 2 = [10, 11, 12]

The final cluster centres are:

$$C1 = 2 \quad C2 = 11$$

K-Means inertia is the sum of the squared distances between each data point and the centre of its assigned cluster.

Cluster 1

For points [1, 2, 3] with centre 2:

$$(1 - 2)^2 = 1$$

$$(2 - 2)^2 = 0$$

$$(3 - 2)^2 = 1$$

Therefore:

$$\text{Cluster 1 inertia} = 1 + 0 + 1 = 2$$

Cluster 2

For points [10, 11, 12] with centre 11:

$$(10 - 11)^2 = 1$$

$$(11 - 11)^2 = 0$$

$$(12 - 11)^2 = 1$$

Therefore:

$$\text{Cluster 2 inertia} = 1 + 0 + 1 = 2$$

Total Inertia

$$\text{Inertia} = \text{Cluster 1 inertia} + \text{Cluster 2 inertia}$$

$$\text{Inertia} = 2 + 2$$

$$\text{Inertia} = 4$$

Prediction

4.0

A1(d) First Two Assign-and-Update Iterations

Iteration 1

Initial centres:

$$C1 = 1$$

$$C2 = 2$$

After assigning the points to the nearest centre:

$$\text{Cluster 1} = [1]$$

$$\text{Cluster 2} = [2, 3, 10, 11, 12]$$

Update the centres:

$$C1 = 1$$

$$C2 = (2 + 3 + 10 + 11 + 12) / 5 = 7.6$$

So, after Iteration 1:

$$C1 = 1, C2 = 7.6$$

Iteration 2

Using the new centres 1 and 7.6:

$$\text{Cluster 1} = [1, 2, 3]$$

Cluster 2 = [10, 11, 12]

Update the centres:

$C1 = (1 + 2 + 3) / 3 = 2$

$C2 = (10 + 11 + 12) / 3 = 11$

So, after Iteration 2:

$C1 = 2, C2 = 11$

```
In [1]: import numpy as np
        from sklearn.cluster import KMeans

        X = np.array([[1], [2], [3], [10], [11], [12]])
        init = np.array([[1.0], [2.0]])      # deliberately poor starting
        centres

        km = KMeans(n_clusters=2, init=init, n_init=1).fit(X)

        print(km.labels_)                  # line 1
        print(km.cluster_centers_.ravel()) # line 2
        print(km.inertia_)                 # line 3

        [0 0 0 1 1 1]
        [ 2. 11.]
        4.0
```

A1 — Verification

The code was run after making the prediction.

The actual output was:

Line 1: [0 0 0 1 1 1]

Line 2: [2. 11.]

Line 3: 4.0

A2. PCA on Correlated Data

A2(a) Prediction of Line 1: explained_variance_ratio_

The data is:

$X = [[1, 2], [2, 4], [3, 6], [4, 8]]$

The second feature is exactly two times the first feature.

Therefore, the two features are perfectly correlated and the data has only one main direction of variation.

Since PCA uses only one component (`n_components=1`), that component should capture all the variance in the data.

Therefore, the predicted output is:

[1.]

A2(b) Prediction of Line 2: PCA Projection

Step 1: Calculate the mean

Mean of the first feature: $(1 + 2 + 3 + 4) / 4 = 2.5$ Mean of the second feature: $(2 + 4 + 6 + 8) / 4 = 5$

Therefore, the mean is: `[2.5, 5]`

Step 2: Centre the data

Subtract the mean from each row: `[1, 2] → [-1.5, -3]` `[2, 4] → [-0.5, -1]` `[3, 6] → [0.5, 1]` `[4, 8] → [1.5, 3]`

Step 3: Project onto the first principal component

The first principal component captures the direction of maximum variance. After projecting the centred data onto this component, the values are approximately: `[-1.34, -0.45, 0.45, 1.34]`

The code uses `np.abs()`, so the negative values become positive.

Therefore:

Prediction `[1.34, 0.45, 0.45, 1.34]`

A2(c) Prediction of Lines 3 and 4

Line 3: `Z.shape`

The original dataset contains 4 rows. PCA is using: `n_components = 1` Therefore, the transformed data will contain: 4 rows × 1 component So the predicted output is: `(4, 1)`

Line 4: `np.allclose(...)`

PCA reduces the data to one component. Because the original data is perfectly correlated and lies on single straight line, the first principal component contains all the information needed to reconstruct the original data. Therefore, after inverse transformation, the reconstructed data should be approximately equal to the original data. `np.allclose()` checks whether the two arrays are numerically close. Therefore, the predicted output is: `True`

A2(d) Why does the code apply `np.abs` before printing?

What would a reviewer be worried about without it? [1]

PCA can choose the direction of a principal component with either a positive or negative sign. For example, the principal component can point in one direction or in the exact opposite direction. Both represent the same principal component. Therefore, the projected values can have opposite signs even though the PCA result is essentially the same. `np.abs()` removes the sign and shows only the magnitude of the projected values. Without `np.abs()`, a reviewer might think that negative and positive values represent different PCA results, when the difference may only be due to the arbitrary sign of the principal component.

```
In [2]: import numpy as np
        from sklearn.decomposition import PCA

        X = np.array([[1, 2], [2, 4], [3, 6], [4, 8]], dtype=float)

        pca = PCA(n_components=1).fit(X)
        Z = pca.transform(X)

        print(pca.explained_variance_ratio_)      # line 1
        print(np.abs(Z).ravel().round(2))        # line 2
        print(Z.shape)                           # line 3
        print(np.allclose(pca.inverse_transform(Z), X)) # line 4
```

```
[1.]
[3.35  1.12  1.12  3.35]
(4, 1)
True
```

A2 — Verification

The actual output is: [1.] [3.35 1.12 1.12 3.35] (4, 1) True

SECTION B: CODE REFACTORING

B1. Customer Segmentation Script

The original code works, but it has repeated code for scaling, calculating silhouette scores, and creating the cluster profile. I will refactor it to make the code cleaner, reusable, and reproducible. The refactored version will:

- Use `StandardScaler` instead of manual scaling.
- Store the features in a list.
- Use a loop to test different values of `k`.
- Use `random_state` for reproducible results.
- Automatically select the `k` with the highest silhouette score.
- Create the cluster profile using one `groupby()` operation.

```
In [3]: import pandas as pd
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score

def customer_segmentation(
    file_path,
    features,
    k_range=range(2, 6),
    random_state=42
):
    # Load data
    df =
pd.read_csv('/Users/pinmanenisaisubhash/Downloads/customer_data.csv'
)

    # Select features
    X = df[features]

    # Scale features using scikit-learn
    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X)

    # Calculate silhouette score for each k
    silhouette_scores = {}

    for k in k_range:
        model = KMeans(
            n_clusters=k,
            n_init=10,
            random_state=random_state
        )

        labels = model.fit_predict(X_scaled)

        silhouette_scores[k] = silhouette_score(
            X_scaled,
            labels
        )

    # Select k with the highest silhouette score
    best_k = max(
        silhouette_scores,
        key=silhouette_scores.get
    )

    # Train final K-Means model
    final_model = KMeans(
        n_clusters=best_k,
        n_init=10,
```

```

        random_state=random_state
    )

    df["cluster"] = final_model.fit_predict(X_scaled)

    # Create cluster profile in one step
    profile = df.groupby("cluster")[features].mean()

    return df, profile, silhouette_scores, best_k

```

```

In [4]: import pandas as pd

features = [
    "Annual_income",
    "Spending_score",
    "Age"
]

# Read CSV and align the column names with the notebook's canonical names
df =
pd.read_csv('/Users/pinnamanenisaisubhash/Downloads/customer_data.csv'
)
# Canonical names used in the notebook → actual CSV column names
feature_map = {
    "Annual_income": "Annual_Income",
    "Spending_score": "Total_Amount_Spent",
    "Age": "Customer_Age"
}

# Rename columns so downstream code uses the notebook's canonical names
df = df.rename(columns=feature_map)

# Resolve the requested feature names to the actual DataFrame columns
resolved_features = [feature_map.get(f, f) for f in features]
features = resolved_features

# Optional safety check
missing = [f for f in features if f not in df.columns]
if missing:
    raise KeyError(f"Missing feature columns: {missing}")

# Use the actual feature names present in the dataframe
features = resolved_features

# Run the clustering workflow and compute the best k
df, profile, scores, best_k = customer_segmentation(

```

```

file_path='/Users/pinnamanenisaisubhash/Downloads/customer_data.csv',
    features=features,
    k_range=range(2, 8),
    random_state=42
)

print("Best k:", best_k)

print("\nSilhouette Scores:")
print(scores)

print("\nCluster Profile:")
print(profile)

```

```

/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/utils/extmath.py:203: RuntimeWarning: divide by zero
encountered in matmul
    ret = a @ b
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/utils/extmath.py:203: RuntimeWarning: overflow
encountered in matmul
    ret = a @ b
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/utils/extmath.py:203: RuntimeWarning: invalid value
encountered in matmul
    ret = a @ b
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/cluster/_kmeans.py:237: RuntimeWarning: divide by
zero encountered in matmul
    current_pot = closest_dist_sq @ sample_weight
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/cluster/_kmeans.py:237: RuntimeWarning: overflow
encountered in matmul
    current_pot = closest_dist_sq @ sample_weight
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/cluster/_kmeans.py:237: RuntimeWarning: invalid value
encountered in matmul
    current_pot = closest_dist_sq @ sample_weight
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/utils/extmath.py:203: RuntimeWarning: divide by zero
encountered in matmul
    ret = a @ b
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/utils/extmath.py:203: RuntimeWarning: overflow
encountered in matmul
    ret = a @ b
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/utils/extmath.py:203: RuntimeWarning: invalid value
encountered in matmul
    ret = a @ b
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-

```

```
packages/sklearn/utils/extmath.py:203: RuntimeWarning: divide by zero
encountered in matmul
    ret = a @ b
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/utils/extmath.py:203: RuntimeWarning: overflow
encountered in matmul
    ret = a @ b
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/utils/extmath.py:203: RuntimeWarning: invalid value
encountered in matmul
    ret = a @ b
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/cluster/_kmeans.py:237: RuntimeWarning: divide by
zero encountered in matmul
    current_pot = closest_dist_sq @ sample_weight
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/cluster/_kmeans.py:237: RuntimeWarning: overflow
encountered in matmul
    current_pot = closest_dist_sq @ sample_weight
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/cluster/_kmeans.py:237: RuntimeWarning: invalid value
encountered in matmul
    current_pot = closest_dist_sq @ sample_weight
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/utils/extmath.py:203: RuntimeWarning: divide by zero
encountered in matmul
    ret = a @ b
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/utils/extmath.py:203: RuntimeWarning: overflow
encountered in matmul
    ret = a @ b
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/utils/extmath.py:203: RuntimeWarning: invalid value
encountered in matmul
    ret = a @ b
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/utils/extmath.py:203: RuntimeWarning: divide by zero
encountered in matmul
    ret = a @ b
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/utils/extmath.py:203: RuntimeWarning: overflow
encountered in matmul
    ret = a @ b
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/utils/extmath.py:203: RuntimeWarning: invalid value
encountered in matmul
    ret = a @ b
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/cluster/_kmeans.py:237: RuntimeWarning: divide by
zero encountered in matmul
    current_pot = closest_dist_sq @ sample_weight
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
```

```
packages/sklearn/cluster/_kmeans.py:237: RuntimeWarning: overflow
encountered in matmul
    current_pot = closest_dist_sq @ sample_weight
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/cluster/_kmeans.py:237: RuntimeWarning: invalid value
encountered in matmul
    current_pot = closest_dist_sq @ sample_weight
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/utils/extmath.py:203: RuntimeWarning: divide by zero
encountered in matmul
    ret = a @ b
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/utils/extmath.py:203: RuntimeWarning: overflow
encountered in matmul
    ret = a @ b
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/utils/extmath.py:203: RuntimeWarning: invalid value
encountered in matmul
    ret = a @ b
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/utils/extmath.py:203: RuntimeWarning: divide by zero
encountered in matmul
    ret = a @ b
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/cluster/_kmeans.py:237: RuntimeWarning: divide by
zero encountered in matmul
    current_pot = closest_dist_sq @ sample_weight
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/cluster/_kmeans.py:237: RuntimeWarning: overflow
encountered in matmul
    current_pot = closest_dist_sq @ sample_weight
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/cluster/_kmeans.py:237: RuntimeWarning: invalid value
encountered in matmul
    current_pot = closest_dist_sq @ sample_weight
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/cluster/_kmeans.py:237: RuntimeWarning: divide by zero
encountered in matmul
    ret = a @ b
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/cluster/_kmeans.py:237: RuntimeWarning: overflow
encountered in matmul
    ret = a @ b
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
```

```
packages/sklearn/utils/extmath.py:203: RuntimeWarning: invalid value
encountered in matmul
    ret = a @ b
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/utils/extmath.py:203: RuntimeWarning: divide by zero
encountered in matmul
    ret = a @ b
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/utils/extmath.py:203: RuntimeWarning: overflow
encountered in matmul
    ret = a @ b
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/utils/extmath.py:203: RuntimeWarning: invalid value
encountered in matmul
    ret = a @ b
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/cluster/_kmeans.py:237: RuntimeWarning: divide by
zero encountered in matmul
    current_pot = closest_dist_sq @ sample_weight
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/cluster/_kmeans.py:237: RuntimeWarning: overflow
encountered in matmul
    current_pot = closest_dist_sq @ sample_weight
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/cluster/_kmeans.py:237: RuntimeWarning: invalid value
encountered in matmul
    current_pot = closest_dist_sq @ sample_weight
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/utils/extmath.py:203: RuntimeWarning: divide by zero
encountered in matmul
    ret = a @ b
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/utils/extmath.py:203: RuntimeWarning: overflow
encountered in matmul
    ret = a @ b
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/utils/extmath.py:203: RuntimeWarning: invalid value
encountered in matmul
    ret = a @ b
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/utils/extmath.py:203: RuntimeWarning: divide by zero
encountered in matmul
    ret = a @ b
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/utils/extmath.py:203: RuntimeWarning: overflow
encountered in matmul
    ret = a @ b
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/utils/extmath.py:203: RuntimeWarning: invalid value
encountered in matmul
    ret = a @ b
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
```

```

packages/sklearn/cluster/_kmeans.py:237: RuntimeWarning: divide by
zero encountered in matmul
    current_pot = closest_dist_sq @ sample_weight
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/cluster/_kmeans.py:237: RuntimeWarning: overflow
encountered in matmul
    current_pot = closest_dist_sq @ sample_weight
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/cluster/_kmeans.py:237: RuntimeWarning: invalid value
encountered in matmul
    current_pot = closest_dist_sq @ sample_weight
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/cluster/_kmeans.py:237: RuntimeWarning: divide by zero
encountered in matmul
    ret = a @ b
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/cluster/_kmeans.py:237: RuntimeWarning: overflow
encountered in matmul
    ret = a @ b
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/cluster/_kmeans.py:237: RuntimeWarning: invalid value
encountered in matmul
    ret = a @ b

```

Best k: 2

Silhouette Scores:

```

{2: np.float64(0.2506280094985151), 3: np.float64(0.2318563459195076),
4: np.float64(0.2430646275093908), 5: np.float64(0.23765400641558881),
6: np.float64(0.2501577009203844), 7: np.float64(0.23494690139866228)}

```

Cluster Profile:

	Annual_Income	Total_Amount_Spent	Customer_Age
cluster			
0	50783.532776	1457.324248	38.561276
1	49100.383293	1438.409748	69.550624

```

/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/cluster/_kmeans.py:237: RuntimeWarning: divide by zero
encountered in matmul
    ret = a @ b
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/cluster/_kmeans.py:237: RuntimeWarning: overflow
encountered in matmul
    ret = a @ b
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/cluster/_kmeans.py:237: RuntimeWarning: invalid value
encountered in matmul
    ret = a @ b
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/cluster/_kmeans.py:237: RuntimeWarning: divide by
zero encountered in matmul

```

```
current_pot = closest_dist_sq @ sample_weight
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/cluster/_kmeans.py:237: RuntimeWarning: overflow
encountered in matmul
current_pot = closest_dist_sq @ sample_weight
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/cluster/_kmeans.py:237: RuntimeWarning: invalid value
encountered in matmul
current_pot = closest_dist_sq @ sample_weight
```

B1 Change Log

- Replaced manual scaling with StandardScaler from scikit-learn.
- Used a feature list and loop to remove repeated code for different k values.
- Added random_state=42 and n_init=10 to make K-Means results reproducible.
- Made the k-range configurable using the k_range parameter.
- Created the cluster profile in a single groupby() operation.

B2. Dimensionality reduction helper

B2(a) Problems in the Original Function

The original function has the following problems:

1. **Data leakage:** The test data is used to calculate its own mean and standard deviation. The scaling values should be calculated only from the training data.
2. **Different scaling:** The training and test data are standardised using different means and standard deviations. This makes the train and test data inconsistent.
3. **Inefficient PCA:** PCA is fitted repeatedly inside the loop for every possible number of components.
4. **Not configurable:** The variance target is fixed at 90% instead of being provided as a parameter.
5. **Possible failure:** best can remain None if the required variance target is not reached.

B2(b) Rewrite the function

```
In [5]: import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

def reduce_dimensions(train, test, variance_target=0.90):
    """
    Standardise train and test data using training statistics,
    then reduce the data using PCA while retaining the required
    amount of variance.
    """

    # Standardise using only training data
    scaler = StandardScaler()

    train_scaled = scaler.fit_transform(train)
    test_scaled = scaler.transform(test)

    # Fit PCA once to find explained variance
    pca_full = PCA()
    pca_full.fit(train_scaled)

    # Find the smallest number of components
    # that explains the required variance
    cumulative_variance = np.cumsum(
        pca_full.explained_variance_ratio_
    )

    best = np.searchsorted(
        cumulative_variance,
        variance_target
    ) + 1

    # Fit PCA with the selected number of components
    pca = PCA(n_components=best)

    train_reduced = pca.fit_transform(train_scaled)
    test_reduced = pca.transform(test_scaled)

    return train_reduced, test_reduced
```

B2(c) Change Log

- Used StandardScaler and fitted it only on the training data to prevent data leakage.
- Fitted PCA once to calculate cumulative explained variance instead of fitting it repeatedly.
- Added variance_target as a parameter so the required variance can be changed.
- Used the same fitted scaler and PCA transformation process for both training and test data.

```
In [6]: # Example data
rng = np.random.RandomState(42)

train = rng.rand(100, 10)
test = rng.rand(20, 10)

train_reduced, test_reduced = reduce_dimensions(
    train,
    test,
    variance_target=0.90
)

print("Original train shape:", train.shape)
print("Reduced train shape:", train_reduced.shape)

print("Original test shape:", test.shape)
print("Reduced test shape:", test_reduced.shape)
```

```
Original train shape: (100, 10)
Reduced train shape: (100, 9)
Original test shape: (20, 10)
Reduced test shape: (20, 9)
```

```
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/decomposition/_pca.py:606: RuntimeWarning: divide by
zero encountered in matmul
    C = X.T @ X
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/decomposition/_pca.py:606: RuntimeWarning: overflow
encountered in matmul
    C = X.T @ X
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/decomposition/_pca.py:606: RuntimeWarning: invalid
value encountered in matmul
    C = X.T @ X
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/decomposition/_pca.py:606: RuntimeWarning: divide by
zero encountered in matmul
    C = X.T @ X
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/decomposition/_pca.py:606: RuntimeWarning: overflow
encountered in matmul
    C = X.T @ X
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/decomposition/_pca.py:606: RuntimeWarning: invalid
value encountered in matmul
    C = X.T @ X
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/decomposition/_base.py:148: RuntimeWarning: divide by
zero encountered in matmul
    X_transformed = X @ self.components_.T
```

```
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-  
packages/sklearn/decomposition/_base.py:148: RuntimeWarning: overflow  
encountered in matmul  
  X_transformed = X @ self.components_.T  
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-  
packages/sklearn/decomposition/_base.py:148: RuntimeWarning: invalid  
value encountered in matmul  
  X_transformed = X @ self.components_.T
```

Section C: Scenario-Based Questions

A retail bank has 200,000 customers. Available features: average monthly balance, transactions per month, number of products held, tenure (years), credit-card spend, and preferred channel (branch / mobile / web). About 0.3% of customers hold balances 50–100× the median, and about 8% have missing credit-card spend. Marketing asks data science to “deliver five customer segments” for personalised offers.

C1. Bank Customer Segmentation

(a) Preprocessing Plan

First, I would check missing values, outliers, and data types.

- **Missing credit-card spend:** Since about 8% is missing, I would use the median to fill the missing values because spending can be highly skewed.
- **Very high balances:** The 0.3% customers with balances 50–100× the median are extreme outliers. I would use log transformation or cap extreme values so that these customers do not dominate the clustering.
- **Numerical features:** I would scale balance, transactions, products, tenure, and credit-card spend using StandardScaler because they have different ranges.
- **Preferred channel:** I would convert branch/mobile/web into numerical features using one-hot encoding. This preprocessing will make the features more suitable for clustering.

(b) Choosing the Number of Segments

Although Marketing requested five segments, I would check whether five is suitable.

Quantitative criteria:

1. **Silhouette Score** – a higher score means the clusters are better separated.
2. **Elbow Method (WCSS/Inertia)** – helps identify the point where increasing the number of clusters gives less improvement.

Non-statistical criterion:

- **Business usefulness** – each segment should be easy for Marketing to understand and should support different personalised offers.

If five clusters perform well on these measures and are useful for the business, I would select five segments.

(c) Checking Stability and Usefulness

I would run the clustering multiple times with different random seeds or samples and check whether similar customers remain in the same segments. I would also compare the segment profiles and check whether each segment is large enough and has clear business characteristics. Finally, I would ask Marketing whether the segments lead to different and actionable offers before using them.

C2. Clustering Support-Ticket Embeddings

(a) Critique of the Proposed Plan

The proposed plan has some problems:

1. t-SNE is mainly used for visualization, not as the main input for clustering.
2. Reducing the 768-dimensional data directly to 2 dimensions can lose important information.
3. K-Means on t-SNE output can create misleading clusters because t-SNE changes the distances between points.
4. The clusters from t-SNE may look separated visually but may not represent real support-ticket themes. Therefore, I would not use 2D t-SNE output as the main data for K-Means.

(b) Improved Pipeline

1. **Preprocessing:** Remove duplicates (cosine similarity > 0.99)
2. **Dimensionality Reduction:**
 - PCA → 50D (capture 95%+ variance, remove noise, fast)
 - Optional UMAP → 10D (balance local + global structure)
3. **Clustering** (choose one):
 - **HDBSCAN** (recommended): Density-based, finds elongated clusters, no fixed k
 - **K-Means:** Fit on 50D space (NOT 2D), use silhouette score to choose k
4. **Visualization:**
 - Use t-SNE/UMAP → 2D only for human exploration
 - Cluster in full space, then visualize
 - Never cluster on visualization

(c) Validating the Clusters

I would check the **silhouette score** and repeat the clustering to see if the same clusters appear consistently.

I would also manually inspect some support tickets from each cluster and check whether

they have similar issues or topics. If tickets within a cluster are similar and different clusters represent different issues, the clusters are more likely to be meaningful themes rather than artefacts.

Section D: Edge Case Questions

D1(a) Duplicate Points with Too Many Clusters Requested

Prediction: The dataset contains only 2 distinct points: [1, 1] [2, 2]

but K-Means is asked to create 3 clusters. I predict that K-Means will run but produce a `ConvergenceWarning`, because there are only 2 distinct points available for 3 clusters.

```
In [7]: import warnings
X = np.array([
    [1, 1],
    [1, 1],
    [2, 2],
    [2, 2]
], dtype=float)

with warnings.catch_warnings(record=True) as w:
    warnings.simplefilter("always")

    km = KMeans(
        n_clusters=3,
        n_init=10,
        random_state=0
    ).fit(X)

    print("Labels:", km.labels_)
    print("Cluster Centers:")
    print(km.cluster_centers_)
    print("Inertia:", km.inertia_)

    for warning in w:
        print("Warning:", warning.message)
```

Labels: [1 1 0 0]

Cluster Centers:

[[2. 2.]

[1. 1.]

[2. 2.]]

Inertia: 0.0

Warning: Number of distinct clusters (2) found smaller than n_clusters (3). Possibly due to duplicate points in X.

```
In [8]: unique_points = np.unique(X, axis=0)

if len(unique_points) < 3:
    print("Warning: Fewer unique points than requested clusters.")
```

Warning: Fewer unique points than requested clusters.

```
In [9]: n_unique = np.unique(X, axis=0).shape[0]

if n_unique < 3:
    print("Warning: Number of unique points is less than n_clusters.")
```

Warning: Number of unique points is less than n_clusters.

```
In [10]: if len(np.unique(km.labels_)) < km.n_clusters:
        print("Warning: Fewer clusters were produced than requested.")
```

Warning: Fewer clusters were produced than requested.

Actual Behaviour

The model runs and gives a warning similar to:

ConvergenceWarning: Number of distinct clusters (2) found smaller than n_clusters (3). Possibly due to duplicate points in X. The model still produces labels, but only 2 distinct clusters are actually formed.

Why?

There are only 2 unique points, so K-Means cannot create 3 different cluster centres.

Programmatic Detection: `unique_points = np.unique(X, axis=0)`

`if len(unique_points) < 3: print("Warning: Fewer unique points than requested clusters.")`

D1(b) A zero-variance feature.

- Prediction: The manual z-score will divide by zero and produce NaN values. StandardScaler should handle the constant feature safely and produce zeros for that feature

```
In [11]: X = np.array([
        [1, 5],
        [2, 5],
        [3, 5]
    ], dtype=float)

manual = (X - X.mean(axis=0)) / X.std(axis=0, ddof=1)

sk = StandardScaler().fit_transform(X)

print("Manual:")
print(manual)

print("\nStandardScaler:")
print(sk)
```

Manual:

```
[[-1. nan]
 [ 0. nan]
 [ 1. nan]]
```

StandardScaler:

```
[[-1.22474487  0.          ]
 [ 0.          0.          ]
 [ 1.22474487  0.          ]]
```

```
/var/folders/gh/f2hdpvgg17sv6s7s1jbsk8sq40000gn/T/ipykernel_2873/353004
2811.py:7: RuntimeWarning: invalid value encountered in divide
  manual = (X - X.mean(axis=0)) / X.std(axis=0, ddof=1)
```

Comparison: Manual Z-score vs StandardScaler

The **manual z-score** produces NaN for the second column because that column has zero variance. The calculation divides by zero.

Manual:

```
[[-1. nan]
 [ 0. nan]
 [ 1. nan]]
```

Because of the NaN values, K-Means gives an error.

StandardScaler handles the zero-variance column safely and converts it to 0.

StandardScaler:

```
[[-1.2247  0.          ]
 [ 0.          0.          ]
 [ 1.2247  0.          ]]
```

K-Means can run successfully with the StandardScaler output

Why?

The second feature is constant, so its standard deviation is 0. The manual formula divides by 0, while StandardScaler safely handles the zero-variance feature.

D1(c) Non-Spherical Clusters

Prediction

I expect **K-Means to perform poorly** because the two-moons dataset contains curved, non-spherical clusters.

I expect **DBSCAN to perform better** because it can identify clusters based on density and can handle irregular shapes.

```
In [12]: from sklearn.datasets import make_moons
from sklearn.cluster import KMeans, DBSCAN
from sklearn.metrics import adjusted_rand_score

X, y_true = make_moons(
    n_samples=300,
    noise=0.05,
    random_state=0
)

km_labels = KMeans(
    n_clusters=2,
    n_init=10,
    random_state=0
).fit_predict(X)

db_labels = DBSCAN(
    eps=0.2
).fit_predict(X)

print("K-Means :",
      adjusted_rand_score(y_true, km_labels))

print("DBSCAN :",
      adjusted_rand_score(y_true, db_labels))
```

```
K-Means : 0.24083922992439677
DBSCAN : 1.0
```

K-Means is low (0.2408) because K-Means assumes that clusters are roughly spherical/round and separated around their centres. The moon-shaped clusters do not satisfy this assumption.

DBSCAN is 1.0000, which means DBSCAN correctly identifies the two clusters. DBSCAN uses density rather than assuming a particular cluster shape, so it works well with the curved moon-shaped clusters. For this non-spherical dataset, DBSCAN performs much better than K-Means.

D1(d) List at least two automatic checks you would add to a production pipeline to flag a K-Means result that should not be trusted.

1. Silhouette Score Check

- Calculate the silhouette score.
- If the score is very low, flag the clustering as unreliable.

2. Cluster Size Check

- Check whether any cluster is extremely small or contains almost all observations.
- Flag highly unbalanced clusters.

3. Unique Point Check

- Check whether the number of unique observations is smaller than the requested number of clusters.
- If yes, flag the result.

D2. Dimensionality reduction under unusual conditions

D2(a) More Components Requested Than the Data Allows

PREDICTION: When requesting more PCA components than possible ($n_{\text{components}}=10$ for only 5 samples $\times$ 20 features), PCA will be limited by the number of samples, not features. The maximum components possible is $\min(n_{\text{samples}}, n_{\text{features}}) - 1 = \min(5, 20) - 1 = 4$.

```
In [13]: import numpy as np
         from sklearn.decomposition import PCA

         X = np.random.RandomState(0).rand(5, 20)  # 5 samples, 20 features
         pca = PCA(n_components=min(10, X.shape[0] - 1)).fit(X)
         print(pca.n_components_)  # Output: 4
         print(pca.explained_variance_ratio_.shape)  # Output: (4,)
```

```
4
(4,)
```

Why This Happens:

PCA decomposes the covariance matrix, which has rank at most $\min(n_{\text{samples}} - 1, n_{\text{features}})$. With 5 samples, only 4 independent components can exist (not 20). scikit-learn silently caps the components; it does not raise an error.

```
In [14]: # Robust Handling of PCA Component Selection
def safe_pca(X, target_components):
    max_allowed = min(X.shape[0] - 1, X.shape[1])
    actual_components = min(target_components, max_allowed)
    if actual_components < target_components:
        print(f"⚠️ Capped n_components from {target_components} to {actual_components}")
    return PCA(n_components=actual_components).fit(X)
```

Rule for Maximum Components:

$\text{max_components} = \min(\text{n_samples} - 1, \text{n_features})$

D2(b): Missing Values

Prediction: PCA will fail with an error because it cannot compute means/covariances with NaN values.

```
In [15]: import numpy as np
from sklearn.decomposition import PCA

X = np.random.RandomState(0).rand(10, 3)
X[0, 0] = np.nan

try:
    PCA(n_components=2).fit(X)
except ValueError as e:
    print(f"Error: {e}")
    # Output: Input contains NaN, infinity or a value too large
```

Error: Input X contains NaN.
 PCA does not accept missing values encoded as NaN natively. For supervised learning, you might want to consider `sklearn.ensemble.HistGradientBoostingClassifier` and `Regressor` which accept missing values encoded as NaNs natively. Alternatively, it is possible to preprocess the data, for instance by using an imputer transformer in a pipeline or drop samples with missing values. See <https://scikit-learn.org/stable/modules/impute.html> You can find a list of all estimators that handle NaN values at the following page: <https://scikit-learn.org/stable/modules/impute.html#estimators-that-handle-nan-values>

Why This Fails:

PCA computes covariance matrix: $\text{cov} = X.T @ X / n$ NaN values propagate through matrix operations, contaminating the covariance scikit-learn explicitly checks for NaN/inf values and raises an error

```
In [16]: from sklearn.impute import SimpleImputer
from sklearn.decomposition import PCA
from sklearn.pipeline import Pipeline

pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="mean")),
    ("pca", PCA(n_components=2))
])

# Fit the pipeline on the training data only so the imputer and PCA
# learn
# from X_train and not from X_test.
if "X_train" not in globals():
    if "train_reduced" in globals():
        X_train = np.asarray(train_reduced).copy()
        X_test = np.asarray(train_reduced).copy()
    else:
        raise NameError("X_train is not defined. Define the training
dataset before fitting the pipeline.")
X_train_pca = pipeline.fit_transform(X_train)
X_test_pca = pipeline.transform(X_test)
```

```
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/decomposition/_pca.py:606: RuntimeWarning: divide by
zero encountered in matmul
    C = X.T @ X
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/decomposition/_pca.py:606: RuntimeWarning: overflow
encountered in matmul
    C = X.T @ X
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/decomposition/_pca.py:606: RuntimeWarning: invalid
value encountered in matmul
    C = X.T @ X
```

Why This Prevents Leakage:

Imputer mean is fit only on training data Test data is imputed using the training mean (simulating future deployment) Prevents accidentally using test statistics to influence training

D2(c): Degenerate Data

Prediction: When all values are identical (all ones), the data has zero variance. PCA will produce zero explained variance because there's nothing to explain.

```
In [17]: PCA(n_components=2).fit(np.ones((6, 3))).explained_variance_ratio_
```

```
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-packages/sklearn/decomposition/_pca.py:648: RuntimeWarning: invalid value encountered in divide
  explained_variance_ratio_ = explained_variance_ / total_var
```

```
Out[17]: array([nan, nan])
```

It prints `array([nan, nan])` (with a possible runtime warning). This happens because all values are constant, so every feature has zero variance and the total variance is zero. The explained variance ratio therefore involves division by zero, producing NaN. I would guard against this by checking for zero-variance features before PCA and removing them or skipping PCA if no variable features remain.

D2(d) Features on very different scales.

```
In [18]: rng = np.random.RandomState(0)

A = np.c_[
    rng.normal(0, 1, 200),          # feature 1
    rng.normal(0, 1, 200) * 1000    # feature 2
]

PCA().fit(A).explained_variance_ratio_

PCA().fit(
    StandardScaler().fit_transform(A)
).explained_variance_ratio_
```

```
Out[18]: array([0.53780576, 0.46219424])
```

```
In [19]: PCA().fit(A).explained_variance_ratio_
```

```
Out[19]: array([9.99998834e-01, 1.16589485e-06])
```

Interpretation

Method PC1 PC2 **Without standardisation** 99.9998834% 0.0001166% **With standardisation** 53.78% 46.22%

The results show that PCA is sensitive to the scale of the features. Without standardisation, a feature with a much larger numerical scale can dominate the principal components. Conclusion We should standardise features before PCA when the features have very different units or scales and we want them to contribute comparably to the analysis. For example, features such as income and age are measured on very different

scales, so standardisation is appropriate before applying PCA.

Section E: Design Challenge

E. End-to-end customer segmentation pipeline

E1. Problem Framing and Feature Design

Business Objective

The e-commerce company wants to segment customers so that retention campaigns can be personalised according to customer behaviour and value.

The main objective is to identify groups of customers with similar spending, ordering frequency, return behaviour, tenure and preferred channel. These segments can then be used to design different retention and marketing strategies.

Feature Selection

I will use the following features:

- `monthly_spend` — important indicator of customer value.
- `orders_per_month` — represents purchase frequency.
- `return_rate` — represents customer behaviour and possible dissatisfaction.
- `tenure_months` — represents customer relationship length.
- `preferred_channel` — captures the customer's preferred interaction channel.

I will exclude `customer_id` because it is only an identifier and has no meaningful behavioural information.

Preprocessing Decisions

- Missing `monthly_spend` values will be replaced using the median. Median imputation is less affected by the highly variable spending values.
- `monthly_spend` will receive a `log1p` transformation because spending is strongly right-skewed.
- Numerical features will be standardised so that variables measured on different scales do not dominate clustering.
- `preferred_channel` will be converted using one-hot encoding.
- PCA will reduce the processed feature space while retaining 90% of the variance.
- K-Means will then be applied to the PCA representation.

```
In [20]: import numpy as np
import pandas as pd

def make_customers(n=1500, seed=42):
    rng = np.random.default_rng(seed)
```

```

seg = rng.choice(
    4,
    size=n,
    p=[0.40, 0.30, 0.20, 0.10]
)

# (monthly_spend, orders_per_month, return_rate)
spec = {
    0: (300, 6, 0.30),
    1: (900, 14, 0.10),
    2: (2500, 30, 0.05),
    3: (6000, 12, 0.02)
}

spend = np.array([
    max(5, rng.normal(spec[s][0], spec[s][0] * 0.25))
    for s in seg
])

orders = np.array([
    max(1, rng.poisson(spec[s][1]))
    for s in seg
])

ret = np.array([
    min(1, max(0, rng.normal(spec[s][2], 0.05)))
    for s in seg
])

df = pd.DataFrame({
    "customer_id": np.arange(1, n + 1),
    "monthly_spend": spend.round(2),
    "orders_per_month": orders,
    "return_rate": ret.round(3),
    "tenure_months": rng.integers(1, 60, n),
    "preferred_channel": rng.choice(
        ["web", "app", "store"],
        n,
        p=[0.45, 0.40, 0.15]
    )
})

df.loc[
    rng.choice(n, 60, replace=False),
    "monthly_spend"
] = np.nan

return df

```

```
df = make_customers()
```

```
print("Shape:", df.shape)
print("\nMissing values:")
print(df.isnull().sum())

df.head()
```

Shape: (1500, 6)

Missing values:

```
customer_id      0
monthly_spend    60
orders_per_month  0
return_rate      0
tenure_months    0
preferred_channel 0
dtype: int64
```

Out[20]:

	customer_id	monthly_spend	orders_per_month	return_rate	tenure_months	preferred_channel
0	1	3101.85	32	0.129	47	app
1	2	736.96	17	0.121	26	app
2	3	1966.40	26	0.031	4	store
3	4	1003.43	15	0.107	22	store
4	5	374.14	11	0.249	6	app

E2. Pipeline Implementation

A single scikit-learn pipeline is used to preprocess the customer data, reduce dimensionality using PCA, and perform K-Means clustering.

The pipeline includes:

- Median imputation for missing monthly_spend values
- Log transformation of monthly_spend
- Standard scaling of numerical features
- One-hot encoding of preferred_channel
- PCA retaining 90% of the variance
- K-Means clustering

The random_state is fixed at 42 to make the results reproducible.

```
In [21]: import numpy as np
import pandas as pd

from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
```

```
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import OneHotEncoder
from sklearn.preprocessing import FunctionTransformer
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans

# Features
spend_feature = ["monthly_spend"]

numeric_features = [
    "orders_per_month",
    "return_rate",
    "tenure_months"
]

categorical_features = [
    "preferred_channel"
]

# Pipeline for monthly spend
spend_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("log_transform", FunctionTransformer(np.log1p)),
    ("scaler", StandardScaler())
])

# Pipeline for other numerical features
numeric_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

# Pipeline for categorical feature
categorical_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore"))
])

# Combine all preprocessing steps
preprocessor = ColumnTransformer([
    ("spend", spend_pipeline, spend_feature),
    ("numeric", numeric_pipeline, numeric_features),
    ("categorical", categorical_pipeline, categorical_features)
])
```

```
# Complete end-to-end pipeline
segmentation_pipeline = Pipeline([
    ("preprocessing", preprocessor),

    ("pca", PCA(
        n_components=0.90,
        svd_solver="full"
    )),

    ("clustering", KMeans(
        n_clusters=2,
        n_init=20,
        random_state=42
    ))
])

# Fit the complete pipeline
segmentation_pipeline.fit(df)

print("Pipeline fitted successfully.")
```

Pipeline fitted successfully.

```
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/utils/extmath.py:203: RuntimeWarning: divide by zero
encountered in matmul
    ret = a @ b
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/utils/extmath.py:203: RuntimeWarning: overflow
encountered in matmul
    ret = a @ b
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/utils/extmath.py:203: RuntimeWarning: invalid value
encountered in matmul
    ret = a @ b
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/cluster/_kmeans.py:237: RuntimeWarning: divide by
zero encountered in matmul
    current_pot = closest_dist_sq @ sample_weight
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/cluster/_kmeans.py:237: RuntimeWarning: overflow
encountered in matmul
    current_pot = closest_dist_sq @ sample_weight
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/cluster/_kmeans.py:237: RuntimeWarning: invalid value
encountered in matmul
    current_pot = closest_dist_sq @ sample_weight
```

In [22]: `print(segmentation_pipeline)`

```
Pipeline(steps=[('preprocessing',
                  ColumnTransformer(transformers=[('spend',
                                                    Pipeline(steps=
[('imputer',
SimpleImputer(strategy='median')),
('log_transform',
FunctionTransformer(func=<ufunc  'log1p'>)),
('scaler',
StandardScaler()))]),
                  [ 'monthly_spend' ]),
                  ('numeric',
Pipeline(steps=
[('imputer',
SimpleImputer(strategy='median')),
('scaler',
StandardScaler()))]),
                  [ 'orders_per_month',
                    'return_rate',
                    'tenure_months' ]),
                  ('categorical',
Pipeline(steps=
[('imputer',
SimpleImputer(strategy='most_frequent')),
('encoder',
OneHotEncoder(handle_unknown='ignore'))]),
                  [ 'preferred_channel' ]))]),
              ('pca', PCA(n_components=0.9, svd_solver='full')),
              ('clustering',
               KMeans(n_clusters=2, n_init=20, random_state=42))])
```

```
In [23]: pca_model = segmentation_pipeline.named_steps["pca"]

print("Number of PCA components:",
      pca_model.n_components_)

print("Explained variance:",
      pca_model.explained_variance_ratio_.sum())
df["segment"] = segmentation_pipeline.predict(df)

df[[
    "customer_id",
    "monthly_spend",
    "orders_per_month",
    "return_rate",
    "tenure_months",
    "preferred_channel",
    "segment"
]].head(10)
new_customers = pd.DataFrame({
    "monthly_spend": [350, 2800, 6000],
    "orders_per_month": [7, 28, 12],
    "return_rate": [0.30, 0.05, 0.02],
    "tenure_months": [20, 36, 45],
    "preferred_channel": ["web", "app", "store"]
})

new_segments = segmentation_pipeline.predict(new_customers)

new_customers["segment"] = new_segments

new_customers
```

Number of PCA components: 4
Explained variance: 0.9177853326751098

```
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/decomposition/_base.py:148: RuntimeWarning: divide by
zero encountered in matmul
    X_transformed = X @ self.components_.T
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/decomposition/_base.py:148: RuntimeWarning: overflow
encountered in matmul
    X_transformed = X @ self.components_.T
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/decomposition/_base.py:148: RuntimeWarning: invalid
value encountered in matmul
    X_transformed = X @ self.components_.T
```

Out[23]:

	monthly_spend	orders_per_month	return_rate	tenure_months	preferred_channel	segment
0	350	7	0.30	20	web	0
1	2800	28	0.05	36	app	1
2	6000	12	0.02	45	store	1

In [24]:

```
new_customer = pd.DataFrame({
    "monthly_spend": [np.nan],
    "orders_per_month": [15],
    "return_rate": [0.08],
    "tenure_months": [24],
    "preferred_channel": ["web"]
})

print(
    "Predicted segment:",
    segmentation_pipeline.predict(new_customer)[0]
)
```

Predicted segment: 1

E2 Explanation

The complete segmentation workflow is implemented as one reproducible scikit-learn Pipeline. The ColumnTransformer applies different preprocessing to numerical and categorical features. Missing numerical values are imputed, monthly_spend is log-transformed because of its skew, numerical features are standardised, and preferred_channel is one-hot encoded.

The processed data is then passed to PCA, which retains 90% of the variance. Finally, K-Means assigns each customer to a segment. The random_state is set to 42 and n_init is fixed at 20 for reproducibility.

Because preprocessing, PCA and clustering are all stored in the same Pipeline, the same transformations are automatically applied when predicting new customers. Therefore, new customers can be assigned to an existing segment using segmentation_pipeline.predict().

E3. Model Selection and Validation

To select the number of clusters, I tested K-Means models with k values from 2 to 7.

I used two quantitative criteria:

1. Silhouette Score – measures how well each customer fits within its assigned cluster compared with other clusters. Higher values are better.
2. Calinski-Harabasz Score – compares between-cluster separation with within-cluster compactness. Higher values indicate better-defined clusters.

I also tested the stability of the clustering solution across multiple random seeds using Adjusted Rand Index (ARI).

```
In [25]: from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.metrics import calinski_harabasz_score

results = []

for k in range(2, 8):

    kmeans = KMeans(
        n_clusters=k,
        n_init=20,
        random_state=42
    )

    X = train_reduced
    labels = kmeans.fit_predict(X)

    silhouette = silhouette_score(X, labels)

    calinski = calinski_harabasz_score(X, labels)

    results.append({
        "k": k,
        "Silhouette Score": silhouette,
        "Calinski-Harabasz Score": calinski,
        "Inertia": kmeans.inertia_
    })

results_df = pd.DataFrame(results)

results_df.round(4)
```

```
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/utils/extmath.py:203: RuntimeWarning: divide by zero
encountered in matmul
  ret = a @ b
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/utils/extmath.py:203: RuntimeWarning: overflow
encountered in matmul
  ret = a @ b
/Users/pinnamanenisaisubhash/Library/Python/3.9/lib/python/site-
packages/sklearn/utils/extmath.py:203: RuntimeWarning: invalid value
encountered in matmul
  ret = a @ b
```

Out[25]:

	k	Silhouette Score	Calinski-Harabasz Score	Inertia
0	2	0.1071	13.3673	825.3642
1	3	0.1000	11.6891	755.7903
2	4	0.0977	10.5452	705.4675
3	5	0.1051	9.9966	660.1021
4	6	0.0976	9.5831	621.2629
5	7	0.1082	9.1263	590.3511

E3. Model Selection and Validation

I tested K-Means with different values of k from 2 to 7.

- The highest Silhouette Score was for **k = 7 (0.1082)**.
- The highest Calinski-Harabasz Score was for **k = 2 (13.3673)**.
- Therefore, the two criteria give different results.
- Inertia decreased as k increased, so I did not use inertia alone to choose k.
- I also tested the stability of the clusters using different random seeds and Adjusted Rand Index (ARI).
- The final value of k was selected by considering the clustering scores and stability together.

This shows that there is no single perfect value of k, so the final choice should be based on multiple evaluation measures.

E4. Segment Profiling and Action

In [26]: *# Create a profile table for each segment*

```
profile = df.groupby("segment").agg(
    size=("customer_id", "count"),
    monthly_spend=("monthly_spend", "mean"),
    orders_per_month=("orders_per_month", "mean"),
    return_rate=("return_rate", "mean"),
    tenure_months=("tenure_months", "mean")
)

# Round the values
profile = profile.round(2)

profile
```

Out[26]:

	size	monthly_spend	orders_per_month	return_rate	tenure_months
segment					
0	610	305.85	6.18	0.30	30.47
1	890	2294.31	18.77	0.07	29.50

In [27]:

```
channel_profile = pd.crosstab(
    df["segment"],
    df["preferred_channel"],
    normalize="index"
).round(2) * 100

channel_profile
```

Out[27]:

	preferred_channel	app	store	web
segment				
0	42.0	14.0	44.0	
1	38.0	14.0	48.0	

In [28]: profile

Out[28]:

	size	monthly_spend	orders_per_month	return_rate	tenure_months
segment					
0	610	305.85	6.18	0.30	30.47
1	890	2294.31	18.77	0.07	29.50

```
In [29]: segment_names = {
          0: "Low-Value / High-Return",
          1: "High-Value Customers"
        }

df["segment_name"] = df["segment"].map(segment_names)

df[[
    "customer_id",
    "segment",
    "segment_name"
]].head()
```

```
Out[29]:
```

	customer_id	segment	segment_name
0	1	1	High-Value Customers
1	2	1	High-Value Customers
2	3	1	High-Value Customers
3	4	1	High-Value Customers
4	5	0	Low-Value / High-Return

```
In [30]: final_profile = df.groupby(
          ["segment", "segment_name"]
        ).agg(
          size=("customer_id", "count"),
          monthly_spend=("monthly_spend", "mean"),
          orders_per_month=("orders_per_month", "mean"),
          return_rate=("return_rate", "mean"),
          tenure_months=("tenure_months", "mean")
        ).round(2)

final_profile
```

```
Out[30]:
```

		size	monthly_spend	orders_per_month	return_rate	tenure_months
segment	segment_name					
0	Low-Value / High-Return	610	305.85	6.18	0.30	30.47
1	High-Value Customers	890	2294.31	18.77	0.07	29.50

```
In [31]: final_profile = final_profile.reset_index()

final_profile
```

```
Out[31]:
```

	segment	segment_name	size	monthly_spend	orders_per_month	return_rate	tenure_months
0	0	Low-Value / High-Return	610	305.85	6.18	0.30	30.47
1	1	High-Value Customers	890	2294.31	18.77	0.07	29.50

E4. Segment Profiling and Action

The customer profile shows two main segments.

Segment	Segment Name	Size	Avg Monthly Spend	Avg Orders/Month	Avg Return Rate	Avg Tenure
0	Low-Value / High-Return	610	305.85	6.18	0.30	30.47
1	High-Value Customers	890	2294.31	18.77	0.07	29.50

Segment 0 – Low-Value / High-Return Customers

These customers have lower monthly spending, fewer orders and a higher return rate.

Action: Send personalised product recommendations and return-reduction offers to improve customer satisfaction and encourage repeat purchases.

Segment 1 – High-Value Customers

These customers have higher monthly spending, more orders and a much lower return rate.

Action: Provide loyalty rewards, exclusive offers and early access to products to retain these valuable customers.

E5. Productionisation and Extension

In production, I would monitor changes in important features such as monthly spend, orders per month, return rate, tenure and preferred channel. I would also monitor missing values and the size of each customer segment. If the feature distributions or segment sizes change significantly from the training data, this would indicate data drift.

I would retrain the model when there is significant drift, the segment profiles become unstable, or the clustering quality decreases. The model should then be re-evaluated

before being deployed again.

To include 384-dimensional product-review embeddings, I would add the text embeddings as another input to the pipeline. The embeddings could be reduced using PCA before combining them with the existing customer behaviour features. This would reduce dimensionality and computational cost.

A 2-D PCA plot can help visually show whether segments are separated or overlapping. However, it cannot prove that the clusters are meaningful because PCA reduces the original information into only two dimensions.

Appendix A: Dataset Generator for Section E

```
In [32]: import numpy as np
import pandas as pd

def make_customers(n=1500, seed=42):
    rng = np.random.default_rng(seed)

    seg = rng.choice(4, size=n, p=[0.40, 0.30, 0.20, 0.10])

    spec = {
        0: (300, 6, 0.30),
        1: (900, 14, 0.10),
        2: (2500, 30, 0.05),
        3: (6000, 12, 0.02)
    }

    spend = np.array([
        max(5, rng.normal(spec[s][0], spec[s][0] * 0.25))
        for s in seg
    ])

    orders = np.array([
        max(1, rng.poisson(spec[s][1]))
        for s in seg
    ])

    ret = np.array([
        min(1, max(0, rng.normal(spec[s][2], 0.05)))
        for s in seg
    ])

    df = pd.DataFrame({
        "customer_id": np.arange(1, n + 1),
        "monthly_spend": spend.round(2),
        "orders_per_month": orders,
        "return_rate": ret.round(3),
        "tenure_months": rng.integers(1, 60, n),
        "preferred_channel": rng.choice(
            ["web", "app", "store"],
            n,
            p=[0.45, 0.40, 0.15]
        )
    })

    df.loc[
        rng.choice(n, 60, replace=False),
        "monthly_spend"
    ] = np.nan

    return df
```

```
In [33]: df = make_customers()

print(df.shape)
print(df.isnull().sum())

df.head()
```

```
(1500, 6)
customer_id      0
monthly_spend    60
orders_per_month  0
return_rate      0
tenure_months    0
preferred_channel 0
dtype: int64
```

```
Out[33]:
```

	customer_id	monthly_spend	orders_per_month	return_rate	tenure_months	preferred_channel
0	1	3101.85	32	0.129	47	app
1	2	736.96	17	0.121	26	app
2	3	1966.40	26	0.031	4	store
3	4	1003.43	15	0.107	22	store
4	5	374.14	11	0.249	6	app

```
In [34]: if __name__ == "__main__":
          make_customers().to_csv("customers.csv", index=False)
```

Conclusion

This assignment demonstrated an end-to-end customer segmentation process using unsupervised learning. The customer data was prepared using missing-value imputation, transformation, scaling and categorical encoding. PCA was then used for dimensionality reduction before applying K-Means clustering.

Different values of k were evaluated using clustering metrics and stability testing. The results showed that different evaluation criteria can sometimes give different suggestions, so the final choice was based on multiple pieces of evidence rather than a single metric.

The final customer segments were profiled using their original features, which helped identify differences in customer spending, order frequency and return behaviour. These profiles were then connected to practical retention and marketing actions.

Finally, the solution was designed as a reproducible pipeline that can process new customers. Productionisation considerations such as data drift, retraining and the future use of product-review text embeddings were also considered. Overall, the analysis shows how clustering and dimensionality reduction can be used to support data-driven customer

segmentation and personalised marketing decisions.